01 OF 15

KUBERNETES

THE FOUNDATION OF MODERN CONTAINER ORCHESTRATION

"Kubernetes doesn't run your app, it runs the complexity behind it."

Why Kubernetes?

Containers solved deployment. But when your application grows, so does the complexity. Managing hundreds of containers manually becomes chaotic, risky and unsustainable.

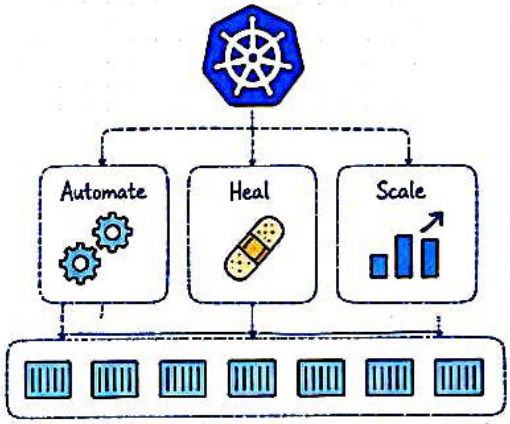
THE PROBLEM

- How do you run hundreds of containers?
- What if a container crashes?
- How do you distribute traffic?
- How do you update without downtime?
- How do you keep everything secure?
- How do you ensure high availability?

Complex
Unreliable
Hard to Scale

THE SOLUTION: KUBERNETES

Kubernetes automates the deployment, scaling, and management of containerized applications.



Simple • Reliable • Scalable

- Automated Management
- Self-Healing
- Scalability
- High Availability
- Rolling Updates
- Service Discovery
- Load Balancing
- Resource Optimization

WHAT IS KUBERNETES?

Kubernetes (K8s) is an open-source container orchestration platform that manages containerized applications across a cluster of machines.

Automates deployment and scaling	Self-heals failed containers	Provides service discovery and load balancing	Enables rolling updates and rollbacks	Optimizes resource utilization

WHY IT MATTERS?

- Powers modern cloud-native applications.
- Used by companies like Google, Netflix, Spotify, Amazon, and many more.
- Industry standard for container orchestration.
- A core skill for DevOps, Cloud, and SRE roles.

USED BY
Google
NETFLIX
Spotify
amazon
& many more...

CONTAINERS vs KUBERNETES

CONTAINERS (What they are)

- Package your application with its dependencies
- Ensure consistency across environments
- Easy to build and run

Containers are the "box"

VS

KUBERNETES (What it does)

- Orchestrates containers
- Manages scaling, networking, storage, and availability
- Ensures your application runs smoothly

Kubernetes is the "orchestrator"

REAL WORLD ANALOGY

A container carries your application → Kubernetes is the ship that manages all containers → It ensures they reach the right destination safely and efficiently

KEY TAKEAWAYS

- Containers are powerful, but managing many manually is hard.
- Kubernetes automates the hard parts so you can focus on building.
- It brings reliability, scalability, and automation to your applications.
- It is the backbone of modern cloud-native infrastructure.

COMMON USE CASES

Cloud-Native Applications	CI/CD Pipeline Deployments	Microservices Architectures	High Traffic Applications	Multi-Cloud & Hybrid Cloud

TRY IT YOURSELF

Run these basic commands after installing kubectl and connecting to a cluster.

```
# Check cluster information
kubectl cluster-info
# Get nodes
kubectl get nodes
# Get all pods in all namespaces
kubectl get pods -A
```

What you'll see:

- Your cluster details
- List of worker nodes
- Running system pods
- Ready to explore more!

PRO TIP
Kubernetes is not just about running containers. It's about operating applications at scale reliably, efficiently, and automatically.

NEXT UP → How does Kubernetes actually control all of this? 02

02 OF 15 KUBERNETES ARCHITECTURE

Think of it as the brain (control plane) that makes decisions and the hands (worker nodes) that do the work.

Kubernetes follows a **Master-Worker architecture** to manage containerized applications.

CONTROL PLANE (The Brain)

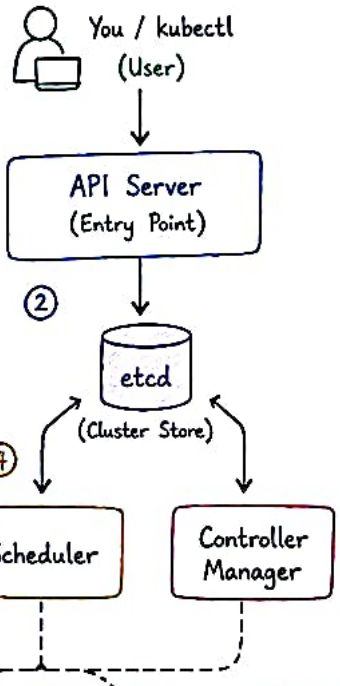
API Server
The entry point for all requests. Validates and processes them.

etcd
The consistent and highly available key-value store that holds all cluster data.

Scheduler
Watches for new Pods without assigned nodes and selects the best node for them.

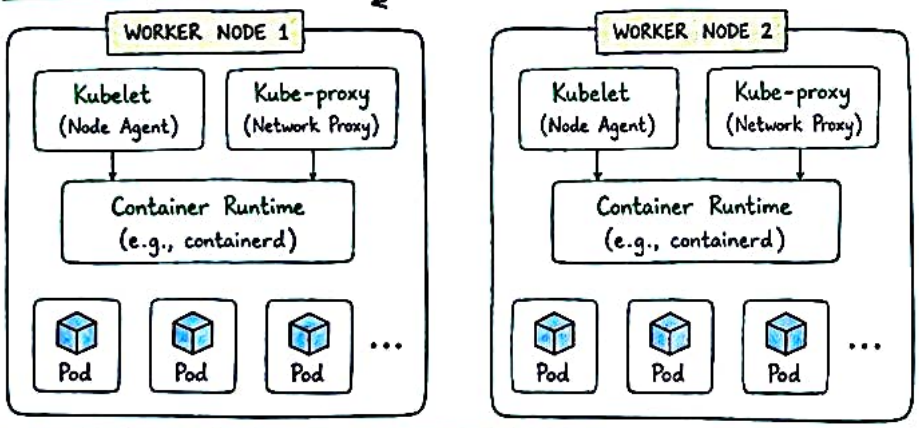
Controller Manager
Runs controllers that ensure the cluster state matches the desired state.

HOW IT WORKS ?



1. You send a request using kubectl.
2. API Server validates the request.
3. Data is stored in etcd.
4. Scheduler finds the best node.
5. Kubelet on the node runs the Pod.

WORKER NODES (The Hands)



★ Kubelet: Ensures containers are running in a Pod.

★ Kube-proxy: Handles network rules and service discovery.

★ Container Runtime: Runs the containers (Docker, containerd, CRI-O, etc.).

KEY TAKEAWAYS

- ✓ Control Plane makes decisions.
- ✓ Worker Nodes run your applications.
- ✓ etcd is the single source of truth.
- ✓ Everything communicates through the API Server.
- ✓ This architecture makes Kubernetes highly scalable, reliable, and resilient.

REAL WORLD EXAMPLE

You order food (Request)
→ API Server takes order
→ etcd keeps the record
→ Scheduler assigns chef (Node)
→ Kubelet tells the chef to cook (Run Pod)
→ You get your food! 🎉

COMMON COMMANDS

```
kubectl get nodes          # List all nodes
kubectl get pods           # List all pods
kubectl get all            # All resources
kubectl describe node <node-name>
                           # Node details
```

PRO TIP

Always start troubleshooting a problem from the **top** (API Server) and move down.

NEXT UP → Where do your applications actually run in Kubernetes? Let's understand **Pods** in the next page!

03 OF 15

PODS

The Smallest Deployable Unit in Kubernetes

A Pod is the smallest unit that can be created and managed in Kubernetes.

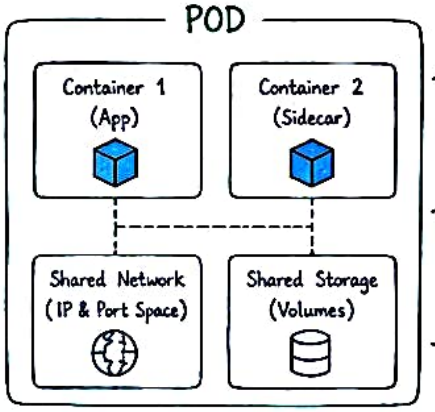
What is a Pod?

A Pod represents a single instance of a running process in your cluster. It can contain one or more containers that share storage, network, and a specification for how to run the containers.

KEY POINTS

- ☆ The smallest deployable unit in Kubernetes.
- ☆ One Pod = One logical host for containers.
- ☆ Containers in a Pod share the same:
  - Network (IP & Port space)
  - Storage (Volumes)
  - Lifecycle
- ☆ Pods are ephemeral. They can be created, destroyed, and recreated.

POD ARCHITECTURE



WHY TO USE MULTIPLE CONTAINERS IN A POD?

- ☑ Sidecar pattern (log collector, proxy, monitoring agent)
- ☑ Helper container (data formatter, certificate updater)
- ☑ Main application + supporting utilities

POD LIFECYCLE

Pending	Running	Succeeded	Failed	Unknown
Pod accepted by the cluster	Pod is bound to a node and containers are running	All containers in Pod completed successfully	All containers terminated and at least one failed	Pod state could not be determined

SINGLE CONTAINER POD EXAMPLE

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
     - containerPort: 80
```

Pod name and labels

Container details inside the Pod

Port exposed by the container

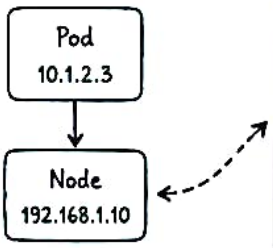
MULTI-CONTAINER POD EXAMPLE

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-app-pod
spec:
  containers:
  - name: app
    image: myapp:latest
    ports:
     - containerPort: 8080
  - name: logger
    image: busybox
    command: ["sh", "-c", "tail -f /var/log/app.log"]
    volumeMounts:
     - name: log-volume
       mountPath: /var/log
  volumes:
  - name: log-volume
    emptyDir: {}
```

Here,

- ☆ Both containers share the same network & storage.
- ☆ The logger container reads the log file generated by the app container.
- ☆ They work together inside the same Pod.

HOW POD GETS AN IP?



- Each Pod gets its own IP.
- IP is from the node's network namespace.
- Containers inside the Pod share the same IP.

COMMON COMMANDS >_

```
kubectl get pods                        # List all pods
kubectl describe pod <pod-name>         # Pod details
kubectl logs <pod-name>                 # View logs
kubectl exec -it <pod-name> -- sh       # Access container
kubectl delete pod <pod-name>           # Delete pod
```

PRO TIP

Pods are designed to be disposable and ephemeral. Use Deployments to manage and run multiple replica Pods.

NEXT UP

Pods run, but they are temporary.
How do we make sure they keep running and heal automatically?

→ NEXT: DEPLOYMENTS

04 OF 15

DEPLOYMENTS

Declarative Updates & Self-Healing for Pods

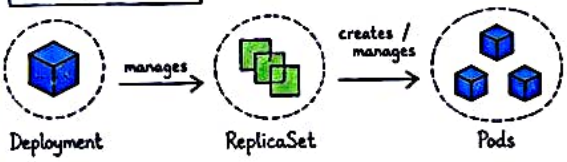
Deployments ensure your application runs the desired number of Pods, always.

WHAT IS A DEPLOYMENT?

A Deployments manages a set of identical Pods and ensures that the desired number of Pods are running at any given time.

It does this by creating and managing ReplicaSets.

CORE CONCEPT



GOAL

Actual State = Desired State (At all times)

If a Pod dies → a new one is created automatically.

HOW IT WORKS

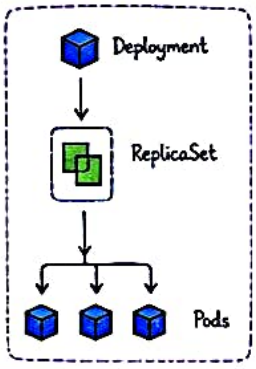
1. You define the desired state in a Deployment (e.g., 3 replicas).
2. Deployment creates a ReplicaSet.
3. ReplicaSet creates the required Pods.
4. If a Pod crashes or is deleted, ReplicaSet creates a new one.
5. If you update the image, Deployment performs a rolling update.

WHY USE DEPLOYMENTS?

- ☑ Self-healing (replaces failed Pods)
- ☑ Rolling updates with zero downtime
- ☑ Rollbacks to previous versions
- ☑ Scaling (up & down) easily
- ☑ Declarative: you define, Kubernetes maintains

DEPLOYMENT COMPONENTS

- Deployment: You define the desired state (replicas, image, strategy, etc.)
- ReplicaSet: Ensures the desired number of Pods are running and healthy.
- Pod: The smallest deployable unit that runs your container(s).



You rarely create ReplicaSets directly. You create a Deployment, and it handles the ReplicaSets.

SAMPLE DEPLOYMENT (YAML)

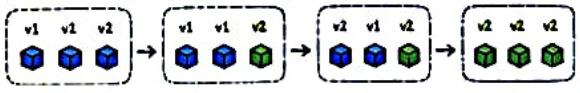
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
```

- How many Pods you want
- How to find which Pods this manages
- Pod blueprint (labels + spec)
- Container information

COMMON OPERATIONS

```
kubectl create deployment nginx --image=nginx:1.25 --replicas=3   # Create
kubectl get deployments                                           # List deployments
kubectl get pods                                                  # See Pods
kubectl describe deployment nginx-deployment                      # Details
kubectl scale deployment nginx-deployment --replicas=5            # Scale up/down
kubectl set image deployment/nginx-deployment nginx=nginx:1.26    # Update image
kubectl rollout status deployment/nginx-deployment                # Check rollout
kubectl rollout history deployment/nginx-deployment               # Revision history
kubectl rollout undo deployment/nginx-deployment                  # Rollback
```

ROLLING UPDATE FLOW



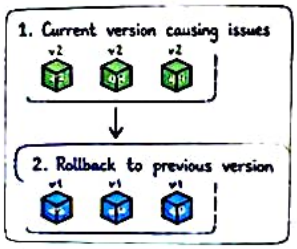
Step 1 — All Pods running version 1

Step 2 — One new Pod (v2) is created, old one is terminated

Step 3 — Update continues gradually

Step 4 — All Pods now running v2

ROLLBACK EXAMPLE



`kubectl rollout undo deployment/nginx-deployment`

Kubernetes rolls back to the previous ReplicaSet (old version).

☑ Fast, safe and reliable.

KEY TAKEAWAYS

- ☆ Deployments manage state, not individual Pods.
- ☆ ReplicaSets ensure the right number of Pods are always running.
- ☆ Great for stateless applications.
- ☆ Rolling updates + rollbacks = safe releases.
- ☆ Keep your app running, even when things fail!

PRO TIP — Use readiness & liveness probes with Deployments to make sure only healthy Pods receive traffic and unhealthy Pods are restarted.

NEXT UP → How do we expose our Pods to other applications or users? Let's learn about Services!

05 OF 15

SERVICES

Stable Networking & Access to Pods

Pods are ephemeral. Services give them a stable IP and DNS name so that other apps can reach them reliably.

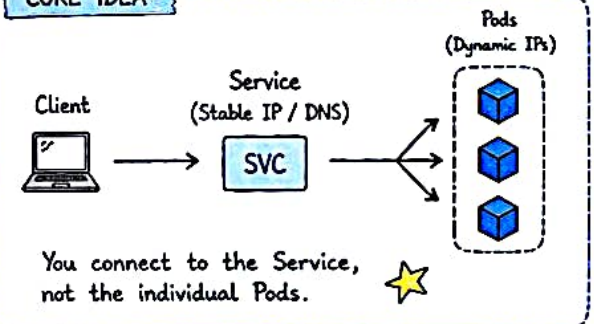
WHY DO WE NEED SERVICES?

- ☑ Pods are dynamic – IPs change when Pods are created or destroyed.
- ☑ Services provide a stable IP and DNS name.
- ☑ They load balance traffic across Pods.
- ☑ They decouple consumers from Pod lifecycle.

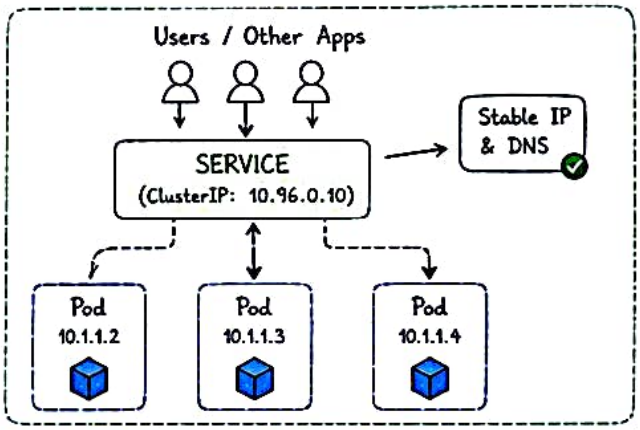
HOW SERVICE WORKS?

1. You create a Service and select Pods using labels.
2. Service gets a stable ClusterIP and DNS name.
3. kube-proxy programs rules on each node.
4. Traffic to Service IP is load balanced to healthy Pods.

CORE IDEA



You connect to the Service, not the individual Pods.



TYPES OF SERVICES

Type	Use Case	Accessible From	How it works	Diagram
ClusterIP (Default)	Internal communication within cluster	Only within the cluster	Exposes the Service on a Cluster-internal IP.	Pod / App → ClusterIP (10.96.x.x) →
NodePort	Expose Service on each Node's IP at a static port	Outside via <NodeIP:NodePort>	Opens a port on every node and forwards traffic to Service.	Outside World → NodeIP:30080 (NodePort) →
LoadBalancer	Expose Service externally using cloud Load Balancer	Outside via Cloud LB IP	Provision cloud LB and route traffic to Service.	Outside World → Cloud LB (External IP) → ClusterIP Service →
ExternalName	Map Service to an external DNS name	Inside cluster resolves to external name	Returns a CNAME record for the specified external name.	Pod / App → ExternalName (my-db.com) → External Service

SAMPLE SERVICE YAML (ClusterIP)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx          # selects Pods with label app=nginx
  ports:
  - protocol: TCP
    port: 80            # Service port
    targetPort: 80      # Pod container port
  type: ClusterIP
```

Service name

Pod selector

Port exposed by Service inside cluster

Port on the Pod container

IMPORTANT NOTES

- ☆ Service DNS format: <svc-name>.<namespace>.svc.cluster.local
- ☆ Default port for NodePort: 30000 – 32767
- ☆ Services are virtually handled by kube-proxy (iptables/ipvs).
- ☆ Service doesn't create or manage Pods.
- ☆ If all Pods behind a Service are down, traffic will not reach any Pod.

COMMON COMMANDS

```
kubectl get svc                 # List all services
kubectl get svc -o wide         # Show details with IP & port
kubectl describe svc <name>     # Detailed information
kubectl delete svc <name>       # Delete a service
```

PRO TIP

Always use labels wisely. Good labels make Services powerful and maintainable!

NEXT UP

Now that Pods are reachable, how do we route external HTTP/HTTPS traffic to our applications?

Let's learn about Ingress!

REAL WORLD SCENARIO Your microservices (auth, user, order) talk to each other using Services, not Pod IPs. Even if Pods restart, communication stays uninterrupted!

NEXT: INGRESS →

06 OF 15

INGRESS

Smart HTTP/HTTPS Routing to Services

If Services expose Pods inside the cluster, Ingress exposes your applications to the WORLD.

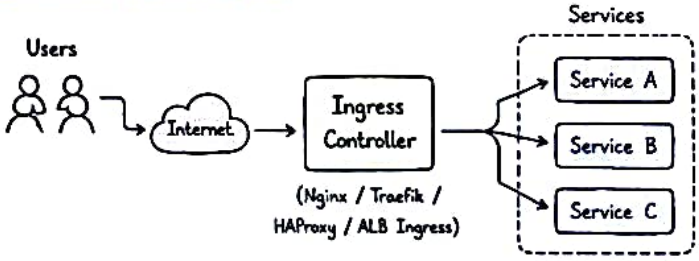
WHAT IS INGRESS?

Ingress is an API object that manages external access to Services in a cluster, typically HTTP and HTTPS. It provides routing rules, based on hostnames and paths, SSL/TLS termination and more.

WHY NOT JUST SERVICE?

- ☑ Service (NodePort / LoadBalancer) exposes only L4. Ingress works at L7 (HTTP/HTTPS).
- ☑ Multiple Services can be exposed using a single IP (based on host/path).
- ☑ Supports SSL/TLS, URL rewriting, authentication, etc.
- ☑ Cleaner, smarter and more powerful routing.

HOW INGRESS WORKS?

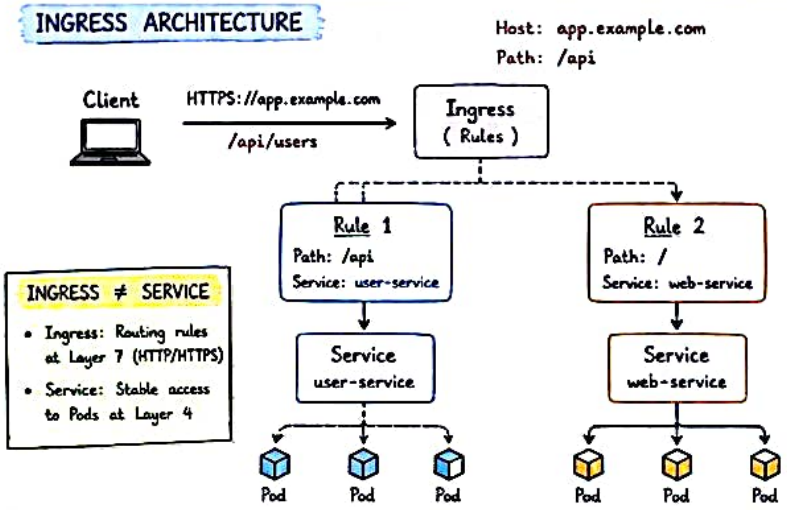


1. Users send HTTP/HTTPS requests.
2. Ingress Controller receives the request.
3. It evaluates Ingress rules (host/path).
4. Traffic is routed to the correct Service.
5. Service sends it to the right Pods.

PRO TIP

Ingress is just rules. Ingress Controller is what implements those rules.

INGRESS ARCHITECTURE



INGRESS ≠ SERVICE

- Ingress: Routing rules at Layer 7 (HTTP/HTTPS)
- Service: Stable access to Pods at Layer 4

INGRESS FEATURES

- ☑ Host-based virtual hosting
- ☑ Path-based routing
- ☑ TLS/SSL termination
- ☑ URL rewrite
- ☑ Rate limiting, auth, etc.
- ☑ Single entry point for apps

WHEN TO USE INGRESS?

- ★ Expose multiple apps using one IP
- ★ Need SSL termination
- ★ Route based on domain or path
- ★ Need advanced HTTP features

SAMPLE INGRESS RULES (YAML)

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress                                   # Name of the Ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /$2   # Rewrite /api/users → /users before sending to Service
spec:
  ingressClassName: nginx                             # Which Ingress Controller will handle this Ingress
  rules:
  - host: app.example.com                             # Domain for which this rule applies
    http:
      paths:
      - path: /api(/|$)(.*)
        pathType: Prefix                              # If path starts with /api, route to user-service
        backend:
          service:
            name: user-service
            port:
              number: 80
      - path: /(.*)                                   # Otherwise, route to web-service
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```

INGRESS vs SERVICE

INGRESS		SERVICE
Works at Layer 7 (HTTP/HTTPS)		Works at Layer 4 (TCP/UDP)
Uses rules (host/path)		Uses labels (selector)
Advanced routing + features		Simple service discovery + load balancing
One IP for many Services		One Service = one IP (inside cluster)
Handled by Ingress Controller		Handled by kube-proxy

COMMON COMMANDS

```
kubectl get ingress                                          # List Ingress
kubectl describe ingress app-ingress                         # Describe Ingress
kubectl get ingress -o wide                                  # Show details
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller # Ingress Controller logs
```

COMMON MISTAKES

- ✘ Forgetting to install Ingress Controller
- ✘ Wrong ingressClassName
- ✘ Missing pathType
- ✘ DNS not pointed to Ingress IP

PRO TIP

Use meaningful hostnames like app.example.com instead of IPs.

REAL WORLD EXAMPLE

company.com
- /api → API Service
- /app → Frontend Service
- /admin → Admin Service

NEXT UP →

How do applications keep configuration and secrets in Kubernetes securely?

Let's learn about ConfigMaps & Secrets!

08 OF 15

VOLUMES & PERSISTENT STORAGE

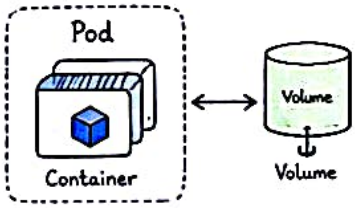
Store Data. Don't Lose It.

Pods are ephemeral. Volumes make data persistent and shareable.

WHY VOLUMES?

- ☑ Data in a Pod is lost when the Pod dies.
- ☑ Volumes persist data beyond Pod lifecycle.
- ☑ Share data between containers in a Pod.
- ☑ Decouple storage from container lifecycle.

HOW IT WORKS

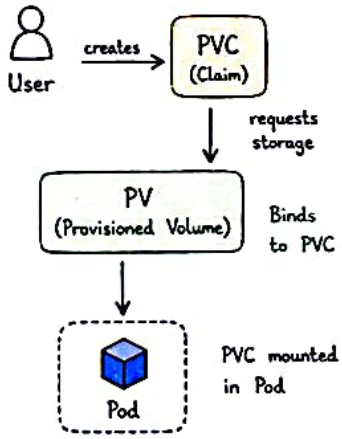


- A Volume is mounted inside the Pod.
- Data written to the Volume is preserved.
- Even if the container restarts, data remains.

TYPES OF VOLUMES

Type	Use Case	Lifetime	Managed By	Notes
emptyDir	Temporary storage, scratch space	As long as Pod lives on a node	Kubernetes	Deleted when Pod is removed.
hostPath	Access files from the node	Until the Pod is removed	Node (Host)	Use with caution (node dependent)
persistentVolume	Cluster storage resource	Until deleted manually	Admin / Kubernetes	Independent of any Pod
persistentVolumeClaim	Request storage from PV	Until PVC is deleted	User	Binds to a PV
configMap / secret	Configuration or sensitive data	Until Pod is removed	Kubernetes	Read-only (mounted as files)

VOLUME FLOW (PV & PVC)



POD USING A VOLUME (EXAMPLE)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
  - name: nginx
    image: nginx
    volumeMounts:
      - name: data
        mountPath: /usr/share/nginx/html
  volumes:
  - name: data
    emptyDir: {}
```

Name of the volume

Where it is mounted inside the container

Define the volume and its type

PERSISTENT VOLUME + PVC (EXAMPLE)

```yaml
# PersistentVolume
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-data
spec:
  capacity:
    storage: 5Gi
  accessModes:
  - ReadWriteOnce
  hostPath:
    path: /data/pv-data
```

```yaml
# PersistentVolumeClaim
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-data
spec:
  accessModes:
   - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
```

User claims the storage

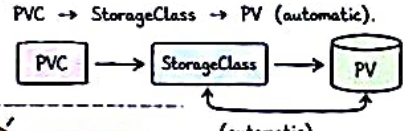
Cluster storage (created by admin)

```yaml
# In Pod
volumes:
- name: data
  persistentVolumeClaim:
    claimName: pvc-data
```

← Pod uses the claim

STORAGE CLASSES (DYNAMIC PROVISIONING)

- ☑ StorageClass allows dynamic provisioning of PV.
- ☑ No need to create PV manually.
- ☑ PVC → StorageClass → PV (automatic).



PRO TIP

- Use PVC + StorageClass for production.
- Choose the right accessModes.
- Back up important data!

COMMON COMMANDS

```
kubectl get pv                    # List PVs
kubectl get pvc                   # List PVCs
kubectl describe pv <name>        # Details
kubectl describe pvc <name>       # Details
kubectl get pods -o wide          # See mounted volumes
```

REAL WORLD SCENARIO

A blog app needs to store user uploads.

→ Use PVC with a StorageClass (e.g., gp3 on AWS / Premium SSD).

→ Data remains even if the Pod or node fails.

REMEMBER: Pods come and go, but your data should stay. Use Volumes & Persistent Storage wisely!

07 OF 15

CONFIGMAPS & SECRETS

Managing Configuration & Sensitive Data

Don't bake configuration or secrets into your images.
Use ConfigMaps for config, Secrets for secrets.

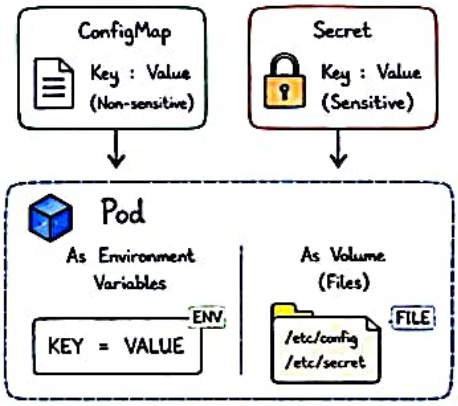
WHAT ARE THEY?

- ConfigMaps store non-sensitive configuration data in key-value pairs.
- Secrets store sensitive information like passwords, tokens, keys, etc.

WHEN TO USE?

- ConfigMap → App settings, feature flags, URLs, environment configs
- Secret → Passwords, API keys, tokens, TLS certs, DB credentials

HOW THEY WORK?



Pods consume these as environment variables or as files mounted in a volume.

KEY POINTS

- ConfigMaps & Secrets are namespaced.
- Secrets are Base64 encoded, NOT encrypted.
- Use RBAC to restrict access.
- Secrets can be mounted as files or env vars.

CONFIGMAP EXAMPLE

Using as Environment Variables

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  APP_MODE: "production"
  LOG_LEVEL: "debug"
  API_URL: "https://api.example.com"
```

Key-Value pairs for non-sensitive config.

In Pod (envFrom)

```yaml
envFrom:
  - configMapRef:
      name: app-config
```

All keys will be available as environment variables.

SECRET EXAMPLE

Using as Environment Variables

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  DB_USER: ZGJhZG1pbg==
  DB_PASSWORD: cGFzc3dvcmQxMjM=
```

Values are Base64 encoded. Decode before using.

In Pod (env)

```yaml
env:
  - name: DB_USER
    valueFrom:
      secretKeyRef:
        name: db-secret
        key: DB_USER
```

Refer specific keys as environment variables.

MOUNT AS VOLUME (FILES)

Both ConfigMaps and Secrets can be mounted as files.

```yaml
volumes:
  - name: config-volume
    configMap:
      name: app-config
containers:
  - name: app
    image: nginx
    volumeMounts:
      - name: config-volume
        mountPath: /etc/config
        readOnly: true
```

Files will be created under /etc/config inside the Pod.

```
/etc/config
├── APP_MODE
├── LOG_LEVEL
└── API_URL
```

CONFIGMAP vs SECRET

Feature	ConfigMap	Secret
Purpose	Non-sensitive config data	Sensitive data
Type	Opaque (text)	Opaque, kubernetes.io/tls, etc.
Encoding	Plain text	Base64 encoded
Security	Not encrypted	More secure (but not encrypted by default)
Use Cases	App configs, URLs, feature flags	Passwords, tokens, TLS certs, API keys

VIEW COMMANDS

```
# ConfigMaps
kubectl get configmaps
kubectl describe configmap app-config
kubectl get configmap app-config -o yaml

# Secrets
kubectl get secrets
kubectl describe secret db-secret
kubectl get secret db-secret -o yaml
```

DECODE SECRET VALUE

Secrets are Base64 encoded.
To decode:

```
echo 'cGFzc3dvcmQxMjM=' | base64 --decode
# Output: password123
```

COMMON MISTAKES

- ✖ Storing secrets in images or code.
- ✖ Using Secrets for non-sensitive data.
- ✖ Not enabling RBAC & restricting access.
- ✖ Forgetting to rotate secrets.

REAL WORLD EXAMPLE

A web app uses:

- ConfigMap → APP_MODE, API_URL, LOG_LEVEL
- Secret → DB_PASSWORD, JWT_SECRET

This keeps the app flexible and secure!

NEXT UP →

Where does data live even if Pods die?
Let's learn about Volumes & Persistent Storage!

09 OF 15

NAMESPACES

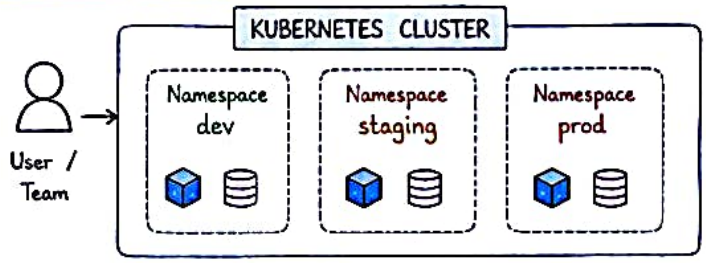
Organize. Isolate. Simplify.

Namespaces divide cluster resources between multiple users, teams or projects.

WHY NAMESPACES?

- ☑ Provide logical isolation within a cluster.
- ☑ Useful for teams, projects or environments.
- ☑ Apply resource quotas and limits.
- ☑ Avoid name collisions between resources.
- ☑ Simplify access control (RBAC).

HOW IT WORKS?



Resources are scoped to their namespace.

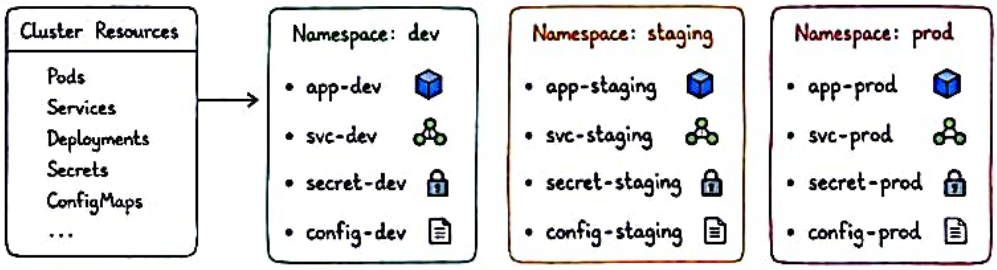
DEFAULT NAMESPACES

- default – For resources with no namespace specified.
- kube-system – System components.
- kube-public – Readable by all users (public info).
- kube-node-lease – Node heartbeat leasing.

EXAMPLE USE CASE

Team / Project	Namespace	Purpose
Development Team	dev	Build and test new features
QA Team	staging	Test before production
Operations Team	prod	Live production workloads

RESOURCE ISOLATION WITH NAMESPACES



NOTE

Resources in one namespace cannot see or access resources in another namespace (unless allowed).

COMMON COMMANDS

```
# List all namespaces
kubectl get ns          # or: kubectl get namespaces
# Create namespace
kubectl create namespace dev
# List resources in a namespace
kubectl get pods -n dev
# Delete namespace
kubectl delete namespace dev
# Set default namespace in context
kubectl config set-context --current --namespace=dev
```

NAMESPACE YAML (EXAMPLE)

```
apiVersion: v1
kind: Namespace
metadata:
  name: dev
  labels:
    purpose: development
    environment: non-prod
```

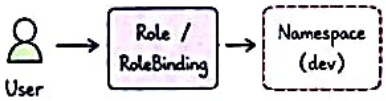
Name of the namespace

Add labels for grouping and filtering.

Helps in organization

RBAC + NAMESPACES

You can restrict access to a namespace using RBAC (Role Based Access Control).



User can access only the resources in the assigned namespace.

PRO TIP

- Use namespaces for different environments (dev, staging, prod).
- Apply ResourceQuotas to control usage.
- Combine with RBAC for secure multi-tenancy.

BEST PRACTICES

- ☆ One namespace per environment or team.
- ☆ Use meaningful names.
- ☆ Avoid using default namespace.
- ☆ Limit cross-namespace access.
- ☆ Clean up unused namespaces.

REAL WORLD SCENARIO

A company runs multiple applications:

→ dev namespace for developers
→ staging for QA testing
→ prod for live users

Keeps everything organized and safe!

REMEMBER:

Namespaces are like rooms in a house.
They keep things organized and prevent chaos!

10 OF 15

HEALTH CHECKS

Keep your Pods Healthy & Your Apps Reliable!

Kubernetes uses Probes to check the health of your containers.

1. LIVENESS PROBE

Checks if the container is ALIVE.

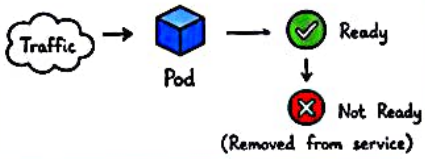
If it fails, Kubernetes restarts the container.

Use Case: App is running but stuck or hung.

2. READINESS PROBE

Checks if the container is READY to serve traffic.

If it fails, Pod is removed from Service endpoints.



Use Case: App is starting up, or dependencies (DB, API) are not ready.

3. STARTUP PROBE

Checks if the application has STARTED successfully.

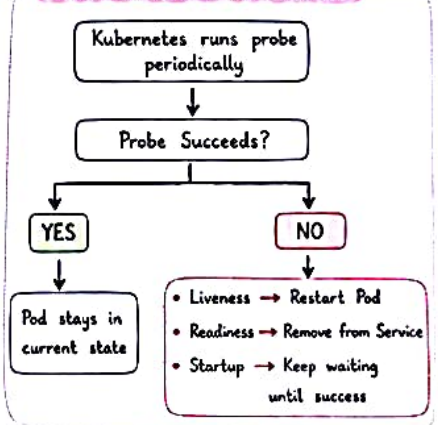
Disables liveness & readiness until it succeeds.

Use Case: App takes long time to start (e.g., big DB, large cache).

PROBE TYPES (WHAT YOU CAN USE)

HTTP GET	Sends HTTP GET request to a path/endpoint. Example: /healthz
EXEC	Runs a command inside the container. Example: cat /tmp/healthy
TCP SOCKET	Checks if a TCP port is open. Example: 8080

HOW PROBES WORK (FLOW)



BEST PRACTICES

- Use Startup probe for slow starting applications.
- Use Readiness probe to avoid sending traffic to unready Pods.
- Use Liveness probe to detect and recover from failures.
- Set initialDelaySeconds, periodSeconds, timeoutSeconds carefully.
- Probes should be fast and lightweight.

EXAMPLE: POD WITH ALL 3 PROBES

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  containers:
  - name: app
    image: my-app:1.0
    livenessProbe:
      httpGet:
        path: /live
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 10
      timeoutSeconds: 2
      failureThreshold: 3
    readinessProbe:
      httpGet:
        path: /ready
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 5
      timeoutSeconds: 2
      failureThreshold: 3
    startupProbe:
      httpGet:
        path: /startup
        port: 8080
      failureThreshold: 30
      periodSeconds: 10
```

→ Restart container if /live fails

→ Remove from Service if /ready fails

→ Wait until /startup succeeds before other probes start working

WHAT HAPPENS WHEN PROBES FAIL?

Probe	Failure Result	Kubernetes Action
Liveness	Container is unhealthy	Restart the container (after failureThreshold)
Readiness	Container not ready	Remove Pod from Service endpoints (no traffic)
Startup	Application not started	Keep checking Startup probe. Other probes are disabled until it succeeds.

REAL WORLD EXAMPLE

- Liveness → Detect if app crashed or deadlocked.
- Readiness → Wait for DB connection, cache warmup, migrations, etc.
- Startup → Useful for apps like Java, .NET, or large ML models.

HEALTHY PODS = HAPPY USERS ☺

Use Probes wisely to build reliable & self-healing applications!

11 OF 15

SCALING

Handle More Load Automatically

Scaling helps your apps handle more users without going DOWN!

1. MANUAL SCALING

You decide how many replicas you want.

```
kubectl scale deployment my-app --replicas=5
```

3 Pods → scale → 5 Pods

Good for predictable or steady workloads.

2. HPA (HORIZONTAL POD AUTOSCALER)

Kubernetes automatically adjusts the number of Pods based on metrics (CPU, Memory, etc).

HPA Controller → Metrics (usage) → Scale Up / Down

- Scales UP when usage goes high
- Scales DOWN when usage goes low
- Keeps your app cost-effective

3. CLUSTER AUTOSCALER (Optional)

Adds or removes Nodes in the cluster automatically based on pending Pods.

Pending Pods → Cluster Autoscaler → Add / Remove Nodes

- Handles node-level scaling
- Useful for large & dynamic workloads
- Requires supported cloud provider

HPA YAML EXAMPLE

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1          # scale at deployment
    kind: Deployment
    name: my-app
  minReplicas: 2                 # maximum Pods
  maxReplicas: 10                # maximum Pods
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70   # if CPU > 70%, scale up
                                 # if CPU < 70%, scale down
```

HOW HPA WORKS

App gets more traffic → CPU/Memory usage increases → HPA detects & scales replicas → More Pods handle the load

HPA checks metrics every 15 seconds (default) and adjusts replicas gradually.

METRICS SUPPORTED BY HPA

Resource Metrics (Built-in)

- CPU utilization
- Memory utilization

Custom / External Metrics

- QPS, RPS
- Queue length
- Application specific metrics

4. RESOURCE REQUESTS & LIMITS

Requests (What a Pod needs)

```
cpu: 200m
memory: 256Mi
```

Helps scheduler place Pods on the right node.

Limits (Max a Pod can use)

```
cpu: 1
memory: 1Gi
```

Prevents a Pod from using too much.

Always set Requests & Limits for better performance and stability.

EXAMPLE: HPA IN ACTION

Time	09:00	09:05	09:10	09:15	09:20
CPU Usage	25%	45%	75%	85%	30%
Replicas	2	2	4	6	3
		Normal load Keep 2 Pods	High load Scale UP	Load decreases Scale DOWN	

USEFUL COMMANDS

```
kubectl get hpa                                  # List all HPA
kubectl describe hpa my-app-hpa                  # Details of HPA
kubectl get pods                                 # Check current replicas
kubectl top pods                                 # CPU/Memory usage
kubectl top nodes                                # Node resource usage
kubectl scale deployment my-app --replicas=3     # Manual scaling
```

BEST PRACTICES

- Set proper requests & limits for all containers.
- Start with reasonable minReplicas (e.g., 2).
- Set maxReplicas based on capacity & cost.
- Use HPA for stateless applications.
- Monitor metrics & tune thresholds.

SCALE SMART, NOT HARD! ☺

Right Pods + Right Metrics = Happy Users

12 OF 15

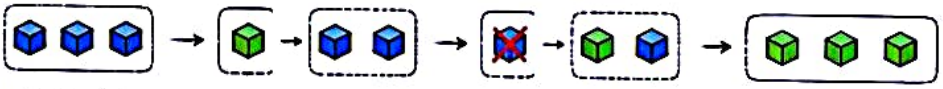
ROLLING UPDATES & ROLLBACKS

Ship New Versions with Zero (or Low) Downtime!

Kubernetes updates your app gradually so users don't even notice!

1. ROLLING UPDATE – HOW IT WORKS

1. Current version (v1)
2. New version (v2) Pod is created
3. One old (v1) Pod is removed
4. Repeat until all Pods are v2



3 Pods (v1) → 1 v2 Pod, 2 v1 Pods → 1 v2 Pod, 2 v1 Pods → 3 Pods (v2)

BENEFITS

- ☑ Zero / Low Downtime
- ☑ Consistent & Controlled updates
- ☑ Automatic health checks
- ☑ Easy rollback if needed
- ☑ Better user experience

ROLLING UPDATE YAML EXAMPLE

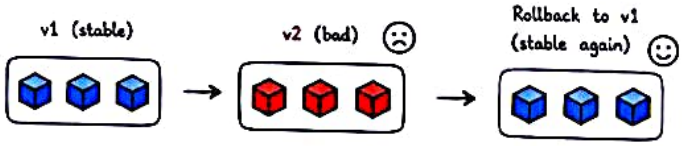
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1   # max Pods unavailable at a time
      maxSurge: 1         # extra Pods during update
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: my-app:v2    # new version
        ports:
         - containerPort: 8080
```

maxUnavailable: 1 → At least 2 Pods remain available

maxSurge: 1 → At most 1 extra Pod is created during update

2. ROLLBACK – IF SOMETHING GOES WRONG

If the new version has issues, rollback to the previous stable version instantly!



ROLLBACK COMMANDS

```
# Rollback to previous revision
kubectl rollout undo deployment my-app

# Rollback to a specific revision
kubectl rollout undo deployment my-app --to-revision=2

# Check rollout history
kubectl rollout history deployment my-app

# See rollout status
kubectl rollout status deployment my-app
```

Kubernetes remembers old ReplicaSets so rollback is super fast!

ROLLOUT HISTORY (EXAMPLE OUTPUT)

```
$ kubectl rollout history deployment my-app

REVISION  CHANGE-CAUSE
1         <none>
2         <none>
3         <none>
4         <none>      <-- current
```

ROLLOUT STATUS (EXAMPLE OUTPUT)

```
$ kubectl rollout status deployment my-app

Waiting for deployment "my-app" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "my-app" rollout to finish: 3 out of 3 new replicas have been updated...
deployment "my-app" successfully rolled out
```

ROLLING UPDATE STRATEGY OPTIONS

- Recreate → All old Pods killed before new ones start.
- RollingUpdate (default) → Gradually replace Pods (recommended).

BEST PRACTICES

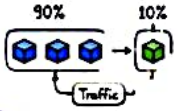
- ☆ Always use RollingUpdate (not Recreate).
- ☆ Set proper readiness & liveness probes.
- ☆ Test in staging before production.
- ☆ Monitor during rollout.
- ☆ Use resource requests/limits for stability.
- ☆ Keep old images for quick rollback.

CANARY / BLUE-GREEN (ADVANCED)

Instead of updating all Pods, send traffic to new version slowly (Canary) or switch all at once (Blue-Green).

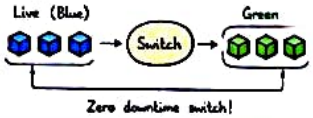
CANARY

Send small % of traffic to new version first.

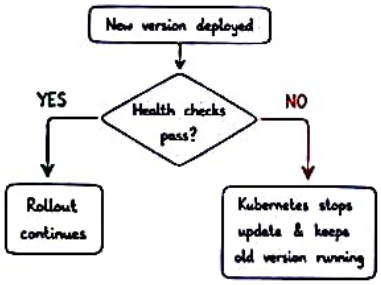


BLUE-GREEN

Run two environments (Blue & Green)



UPDATE FAILURE? WHAT HAPPENS?



QUICK COMMANDS CHEAT SHEET

```
kubectl rollout status deployment <name>                  # check rollout status
kubectl rollout history deployment <name>                 # view history
kubectl rollout undo deployment <name>                    # rollback to previous
kubectl rollout undo deployment <name> --to-revision=N    # rollback to specific
kubectl scale deployment <name> --replicas=N              # manual scale
```

REAL WORLD EXAMPLE

You update your app from v1 to v2.

- ☑ Users keep getting responses (no downtime).
- ☑ If v2 has bugs, rollback in seconds.
- ☑ Your app stays available & reliable.

Update with confidence. Rollback with safety. Users stay happy!

13 OF 15

TROUBLESHOOTING

Find Problems Fast, Fix Problems Smart!

Kubernetes gives you everything you need to find & fix issues!

TROUBLESHOOTING PROCESS

1. Observe the problem
2. Identify where it's failing
3. Gather information
4. Find the root cause
5. Fix the issue
6. Verify the fix

ESSENTIAL KUBECTL COMMANDS (YOUR BEST FRIENDS)

```
kubectl get all                          # overview of all resources
kubectl get pods -o wide                 # pods + node info
kubectl describe <resource> <name>       # detailed info & events
kubectl logs <pod-name> [-c container]   # container logs
kubectl exec -it <pod-name> -- sh        # access pod shell
kubectl get events --sort-by='.lastTimestamp'  # recent events
kubectl get nodes                        # node status
kubectl top pods                         # resource usage (needs metrics-server)
```

Tip: Describe + Logs + Events = 80% of troubleshooting!

WHERE TO LOOK?

- **Events** — What happened?
- **Pods** — Are containers running?
- **Logs** — What's inside the app?
- **Network** — Can things talk to each other?
- **Configuration** — Any wrong settings?

COMMON POD PROBLEMS & HOW TO FIX

1. PENDING POD

Symptoms:
Pod is stuck in Pending state.

Pending

Common Causes:
- Not enough resources
- Node selector/affinity mismatch
- Taints on nodes

How to Fix:
- Check events & describe pod
- Check node capacity
- Check taints & tolerations

Useful Commands:
```
kubectl describe pod <pod-name>
kubectl get nodes
```

2. CRASHLOOPBACKOFF

Symptoms:
Pod keeps restarting.

CrashLoopBackOff

Common Causes:
- App error
- Wrong command/args
- Misconfiguration

How to Fix:
- Check logs
- Fix the issue in app/config
- Redeploy

Useful Commands:
```
kubectl logs <pod-name>
kubectl describe pod <pod-name>
```

3. IMAGEPULLBACKOFF

Symptoms:
Pod can't pull the image.

ImagePullBackOff

Common Causes:
- Wrong image name
- Image doesn't exist
- Private registry auth issue

How to Fix:
- Check image name
- Check image pull secrets
- Verify registry access

Useful Commands:
```
kubectl describe pod <pod-name>
```

4. OOMKILLED

Symptoms:
Container is killed due to Out Of Memory.

RIP OOMKilled

Common Causes:
- App using more memory
- Memory limit is too low

How to Fix:
- Increase memory limit
- Optimize the application

Useful Commands:
```
kubectl logs <pod-name>
kubectl describe pod <pod-name>
```

CHECK EVENTS (FIRST PLACE TO LOOK)

```
kubectl get events --sort-by='.lastTimestamp'
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
10s	Warning	FailedScheduling	pod/my-app	0/3 nodes are available: Insufficient memory.
5s	Warning	BackOff	pod/my-app	Back-off restarting failed container

Events tell you WHAT happened and WHY!

CHECK POD DETAILS (DETAILED INFO)

```
kubectl describe pod <pod-name>
```

- General Info → Name, Namespace, Node, IP
- Containers → Image, State, Ready, Restarts
- Conditions → PodScheduled, Ready, Initialized
- Events → All related events for the pod

CHECK LOGS (WHAT'S INSIDE?)

```
kubectl logs <pod-name>                  # all containers
kubectl logs <pod-name> -c <container>   # specific container
kubectl logs -f <pod-name>               # follow logs
```

Logs = What your application is saying!

NETWORK TROUBLESHOOTING

- Can pod reach other pod?
  `kubectl exec -it <pod> -- ping <other-pod-ip>`
- Can pod reach service?
  `kubectl exec -it <pod> -- curl http://<service>:<port>`
- Check DNS
  `kubectl exec -it <pod> -- nslookup <service>`
- Check endpoints
  `kubectl get endpoints <service-name>`

NODE LEVEL CHECKS

```
kubectl get nodes
```

- Check node status
- Check allocatable resources
- Check conditions (Memory, DiskPressure, PIDPressure)
- Describe node for more info

```
kubectl describe node <node-name>
```

QUICK TROUBLESHOOTING CHEAT SHEET

1. Check pods	`kubectl get pods`
2. Describe pod	`kubectl describe pod <pod-name>`
3. Check events	`kubectl get events --sort-by='.lastTimestamp'`
4. Check logs	`kubectl logs <pod-name>`
5. Exec into pod	`kubectl exec -it <pod-name> -- sh`
6. Check resources	`kubectl top pods`

BEST PRACTICES

- Check EVENTS first.
- Use DESCRIBE to get full picture.
- Check LOGS next.
- Verify NETWORK if app can't communicate.
- Fix one thing at a time.
- Keep calm & read the error message!

REMEMBER

Kubernetes won't hide problems.
It gives you all the tools.
You just need to know WHERE to look!

OBSERVE → ANALYZE → IDENTIFY → FIX → VERIFY
That's how you become a Kubernetes PRO!

14 OF 15

REAL-WORLD KUBERNETES ARCHITECTURE

All the Pieces Working Together!

This is how Kubernetes runs real applications in production!

1. CI/CD FLOW

Code pushed to Git (GitHub)

CI / CD — Build, test and create Docker image

Container Registry (Docker Hub / ECR / ACR)

Deploy to Kubernetes

2. KUBERNETES CLUSTER (PRODUCTION)

Control Plane

- API Server (Entry point)
- Scheduler (Places Pods)
- etcd (Stores state)

Worker Node

Pod | Pod | Pod

kubelet | kube-proxy

Worker Node

Pod | Pod | Pod

kubelet | kube-proxy

3. TRAFFIC FLOW

Users

Load Balancer

Ingress (NGINX / ALB)

Service (ClusterIP)

Pod | Pod | Pod

Service (ClusterIP)

Pod | Pod | Pod

4. APP DEPLOYMENT

- Application runs in Pods
- Managed by Deployments
- Self-healing & auto-recovery
- Rolling updates for zero downtime

Deployment (simplified):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-app
  template:
    metadatt:
      labels:
        app: my-app
```

5. STORAGE & CONFIGURATION

ConfigMap

- Non-sensitive config
- Environment values

Secret

- Sensitive data
- Passwords, keys

Persistent Volume (PVC)

- Durable storage for stateful apps
- Data survives Pod restart

6. SCALING & RELIABILITY

- HPA scales Pods based on CPU/Memory
- Probes keep applications healthy
- Scheduler spreads Pods across nodes
- Cluster Autoscaler adds/removes nodes

HPA (simplified):

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
```

7. KEY BENEFITS

- High availability
- Auto scaling
- Self healing
- Zero / Low downtime
- Efficient resource usage
- Easy rollbacks
- Cloud + on-prem support

8. REAL WORLD USE CASES

- Web applications (E-commerce, SaaS)
- Databases (Stateful apps)
- AI / ML workloads
- Microservices
- Hybrid / Multi-cloud setups

9. TOOLS ECOSYSTEM

- Helm (Package manager)
- Argo CD (GitOps)
- Prometheus (Monitoring)
- Grafana (Visualization)
- ELK (Logging)

10. BIG PICTURE

Kubernetes brings everything together...

Containerize → Deploy → Scale → Heal → Monitor

and keeps your applications running reliably at any scale!

From Development to Production...
Kubernetes Makes Applications Stronger, Smarter and Always Available!

15 OF 15

KUBERNETES MENTAL MODEL

Think in Systems, Not in Objects!

You don't need to memorize everything. Understand how the pieces connect. That's how you become a K8s Pro!

THE CORE FLOW (Big Picture)

1. **CODE** — You write the code
2. **IMAGE** — Build & push Docker image
3. **POD** — Smallest deployable unit (1+ containers)
4. **DEPLOYMENT** — Manages Pods (replicas, updates, self-healing)
5. **SERVICE** — Stable endpoint to access Pods
6. **INGRESS** — HTTP/HTTPS entry point (rules, paths)
7. **USERS** — Users access your app

EVERYTHING THAT SUPPORTS THE FLOW

CONFIGMAPS
- Store non-sensitive configuration
- Environment, files, properties

SECRETS
- Store sensitive data
- Passwords, tokens, API keys, TLS certs

PERSISTENT VOLUME (PVC)
- Requests storage for your Pod
- Data survives Pod restarts

PROBES
- Liveness → Is app alive?
- Readiness → Is app ready?
- Startup → Give time to start

HPA (SCALING)
- Automatically adjusts replicas based on CPU / Memory / Custom metrics

BEHIND THE SCENES

SCHEDULER — Decides where Pods run

ETCD — Stores all cluster data (key-value)

CONTROLLER MANAGER — Runs control loops (Deployment, ReplicaSet, Node, etc.)

KUBELET — Runs on each node. Makes sure containers are running

CONTAINER RUNTIME — Runs containers (containerd / Docker)

HOW UPDATES WORK (Simplified)

1. Current Version (v1)
2. New Version (v2) created
3. Rolling Update (Gradually replace)
4. All Pods on v2

If something goes wrong → Rollback to v1

TROUBLESHOOTING MENTAL MODEL

1	Is the Pod running?	→ kubectl get pods
2	Why is it not running?	→ kubectl describe pod <pod>
3	Check logs	→ kubectl logs <pod>
4	Check events	→ kubectl get events --sort-by=.lastTimestamp
5	Check resources	→ kubectl top pods / nodes
6	Check network / service	→ kubectl get svc, ep, ingress
7	Still stuck?	→ Take a deep breath

COMMON POD STATUS (What they mean)

- Pending → Still waiting (scheduling, image, volume, etc.)
- Running → Pod is running
- CrashLoopBackOff → App crashed repeatedly
- ImagePullBackOff → Can't pull image
- Completed → Job/Pod finished successfully
- OOMKilled → Out of memory
- Evicted → Node pressure / resource issues
- Terminating → Pod is being deleted

KEY TAKEAWAYS

- [x] Kubernetes is a system.
- [x] Everything is connected.
- [x] Focus on the flow, not individual objects.
- [x] Design for failure, scale & change.
- [x] Observe, understand, then fix.
- [x] Practice by building real things!

YOU WILL GET BETTER BY...

- Building projects
- Breaking things
- Debugging
- Reading docs
- Thinking in systems

Consistency > Talent

Understand how the pieces connect.
You don't need to know everything, just the right things for the right time.
KEEP LEARNING, KEEP BUILDING, KEEP SHARING!